Ciclos de vida. Técnicas de elicitación

Cuando hablamos de validación de requerimientos, debemos tener en claro algunos conceptos que constituyen el desarrollo del *software*.
Debemos identificar los diferentes elementos que componen la validación de requerimientos y tener en cuenta el desarrollo de diferentes tipos de verificación y pruebas que tenemos disponibles.

- Caso de análisis
- Modelo en cascada
- Metodología de elicitación
- Referencias
- Revisión de Módulo

Caso de análisis

Una cadena de cabañas permite realizar la reserva de habitaciones vía web. La persona interesada reservará un tipo de habitación para un periodo determinado que posee una tarifa que varía según la temporada (alta, media o baja). En el momento de la reserva, no deberá abonar nada, pero sí deberá hacerlo cuando la confirme, un mes antes del periodo elegido. La cadena tiene un catálogo con los tipos de cabañas con que cuenta y las tarifas de acuerdo con la temporada, que enviará dos veces por año a los clientes para que estos consulten. El número de cabaña se asigna cuando los clientes llegan al complejo.

La cadena de cabañas se encuentra en toda la Argentina y necesita en cualquiera de las sucursales poder realizar reservas, generar el *check in* y *check out* de cada cliente.

Las cabañas tienen actualmente una tecnología que quedó desactualizada. Tanto el procesamiento de los volúmenes de información como el lenguaje de programación son inadecuados. Cuentan con lenguajes de programación para procesar la información que son diferentes, y esto influye en el funcionamiento de la página.

El Equipo de Sistemas de las cabañas decide que se deben llevar a cabo los siguientes puntos:

- El desarrollo de la página web debe ser rápido y se deben poder ver avances en tiempos cortos utilizando el ciclo de vida que se va a elegir.
- La página web debe poder irse actualizando con diferentes versiones a medida que la tecnología evolucione y que se utilice PHP para el diseño de la página web.
- El *software* que necesita el hotel debe poder recibir la visita de 1000 clientes en dos horas.
- Debe haber un disco rígido para almacenar la información de por lo menos 1 terabyte.
- Se debe desarrollar un sistema de reservas que permita identificar un análisis de riesgos detallado, a partir del cual se identifiquen los elementos que contiene el sistema de reservas y se les otorgue un relevamiento a los responsables del Área de Reservas acerca de la información obtenida en el relevamiento, complementando la información con lo obtenido de 200 empleados que se encuentran relacionados con el Área de Reservas.
- Se necesitan registrar los requerimientos en un documento gráfico detallado y una herramienta gráfica.
- Es preciso que se pueda identificar cada interfaz en forma sencilla y que no falte ningún elemento.
- Las cabañas deben poder registrar modificar consultar, anular e imprimir las reservas.
- Se debe desarrollar un sistema de facturación que permita facturar con un IVA del 25 % para clientes que vienen de Brasil, para el cual es necesario tener la información de 25 personas.
- Es necesario contemplar los riesgos que comprenden el sistema de facturación para conocer las medidas que se van a aplicar.

- Se desarrollará un sistema de movimientos diarios de dinero, para cuya construcción se utilizará el ciclo de vida secuencial por la claridad de los elementos que componen el sistema.
- Se viene desarrollando un sistema de cobranzas donde se están probando los diferentes componentes del sistema con su código correspondiente.
- El sistema de *check in* se encuentra terminando la versión 1.1 para poder probarla en el sistema operativo requerido. Es preciso que este utilice SQL como base de datos y que se pueda compartir recursos en la red que tiene el hotel.
- Una vez esté integrado el sistema de *check in,* es necesario cerciorarse de que funcione bien ante 100 solicitudes de *check in* de 8 a 10 h y que cada registro, consulta e impresión funcionen adecuadamente.
- En el sistema de *check in*, se deben hacer testeos de fallas de datos, fallas de control, fallas de entrada/salida, fallas de interfaz y fallas de gestión de almacenamiento.
- Se necesita que el sistema de reservas sea aprobado por los usuarios para verificar que funcione sin errores.
- Es preciso que la empresa tenga una impresora láser color que funcione bajo el sistema operativo Windows.
- La cadena de cabañas se encuentra distribuida en región norte, región este, región oeste y región sur. Cada región cuenta con cinco hoteles. Es necesario orientar el complejo de cabañas a que pueda elegir el ciclo de vida que considere necesario para el desarrollo de cada sistema y las diferentes técnicas de elicitación para obtener la información correspondiente. Para esto, empezaremos a descubrir el primer ciclo de vida.

CONTINUAR

Modelo en cascada

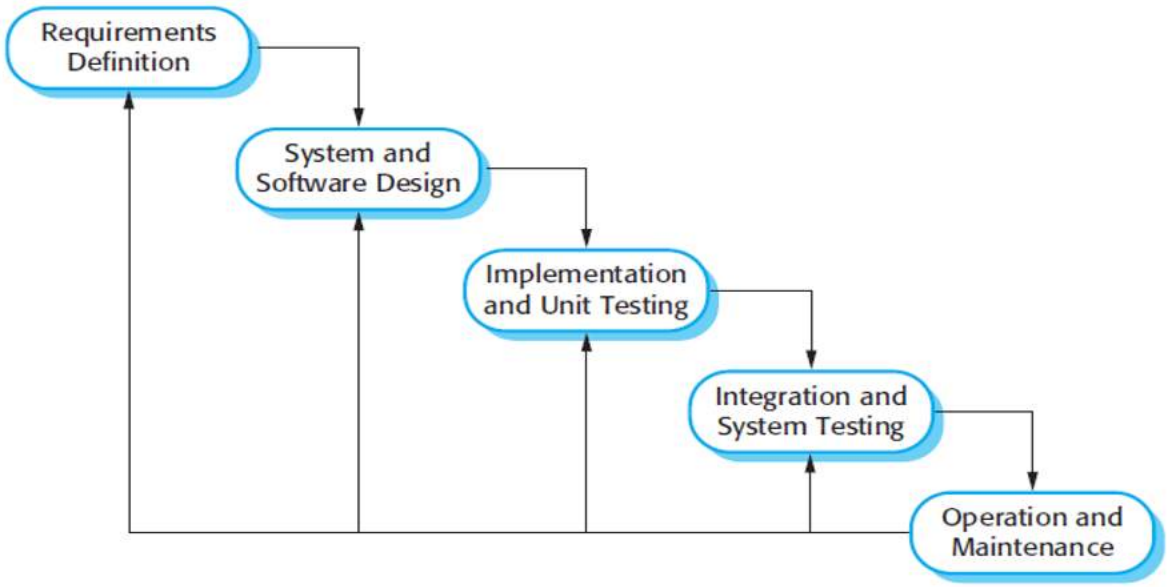
El origen de este modelo de proceso de desarrollo de *software* se publicó y derivó de procesos de ingeniería de sistemas más generales.

Las etapas que lo componen son las siguientes:

Primera etapa: análisis y definición de requerimientos

Los servicios, restricciones y metas del sistema se definen a partir de las consultas que se les hacen a los usuarios.

Figura 1: Análisis y definición de requerimientos



Fuente: [imagen sin título sobre análisis y definición de requerimientos], s. f., https://bit.ly/3fEWhVw

El resultado de estas consultas se traduce y sirve en una especificación del sistema.

(Tengamos en cuenta que vamos a definir lo que son los requerimientos funcionales y los no funcionales).

Veamos una imagen muy conocida:

Figura 2: Requerimientos del usuario



Fuente: adaptación propia con base en [imagen sin título sobre requerimientos del usuario], s. f., https://bit.ly/3hJW4TD

Hay que tener cuidado al interpretar los requerimientos del usuario.

Segunda etapa: diseño del sistema y del *software:*

El proceso de diseño del sistema divide los requerimientos del sistema en *hardware*, *software* y *firmware* según corresponda.

Se establece una arquitectura completa del sistema.

En la etapa de diseño del *software*, se identifican y describen las abstracciones del sistema de software y sus relaciones.

Tercera etapa: implementación y prueba de unidades

Durante esta etapa, el diseño del *software* se lleva a cabo como un conjunto de unidades de programas. La prueba de unidad implica verificar que cada una cumpla con su función.

Cuarta etapa: funcionamiento y mantenimiento

Por lo general, es la etapa más larga del ciclo de vida. El sistema se instala y se pone en funcionamiento práctico.

El mantenimiento no implica corregir errores no descubiertos en las etapas anteriores del ciclo de vida: implica desarrollar un mantenimiento adecuado para que el software funcione adecuadamente.

Siempre se establece que el *software* se entrega con 0 (cero) error.

Ventajas del modelo de cascada

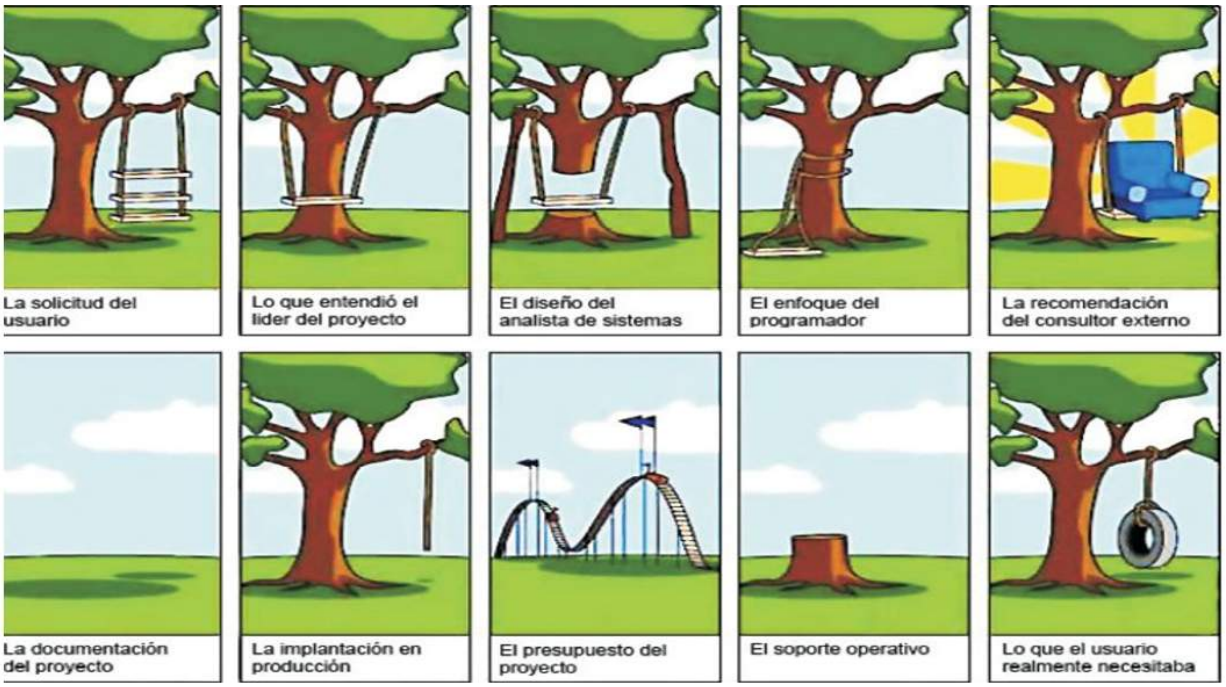
Es un ciclo de vida ideal para cuando se tienen en claro los requerimientos funcionales.

Desventajas del modelo de cascada

- Es rígido, poco.
- Tiene muchas restricciones.
- Es costoso: cualquier error eleva el costo.
- Detectar errores es difícil.
- Se necesita tener en claro los requerimientos al comienzo del proyecto.

Volveremos a recordar y completar la imagen anterior para poder comprender los requerimientos que pide el usuario.

Figura 3: Requerimientos del usuario II

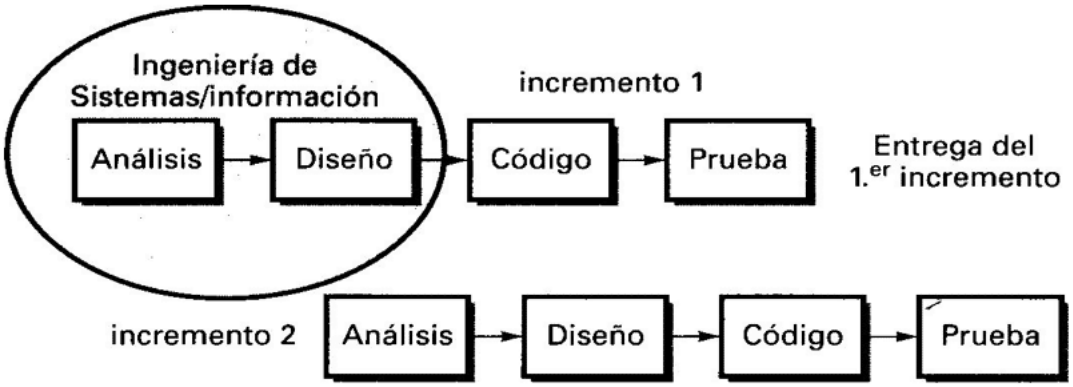


Fuente: [imagen sin título sobre requerimientos del usuario], s. f., https://bit.ly/3hJW4TD

Desarrollo incremental o evolutivo

Se basa en la idea de desarrollar una implementación inicial, luego exponerla a los comentarios del usuario e ir refinando a medida que se van liberando las diferentes versiones, hasta que se llega a un sistema adecuado.

Figura 4: Desarrollo incremental



Fuente: [imagen sin título sobre desarrollo incremental], s. f., https://bit.ly/3fEiCmb

Si prestamos atención al esquema, podemos observar que en el incremento 2 empieza el análisis, con el diseño de la entrega del primer incremento. Si hubiese un incremento 3, se empezaría el análisis a partir del diseño de la entrega del segundo incremento, y así sucesivamente.

Ventajas del modelo incremental o evolutivo

- Se reduce el tiempo de desarrollo inicial, ya que se implementa una funcionalidad parcial.
- Si se detecta un error, se desecha la última iteración.
- Es factible detectar errores.
- Permite escalabilidad.
- No es necesario tener en claro los requerimientos al comienzo del proyecto.
- Se reducen costos.

Dentro del **modelo incremental o evolutivo, tenemos lo siguiente**

Desarrollo exploratorio

- El objetivo del proceso es **trabajar con el cliente para explorar sus requerimientos y entregar el sistema final**.
- El desarrollo empieza con las partes del sistema que se comprenden mejor.
- El sistema evoluciona agregando nuevos atributos propuestos por el cliente.

Prototipos desechables

El objetivo del proceso de desarrollo evolutivo es **comprender los requerimientos del cliente y entonces desarrollar una definición mejorada de los requerimientos para el sistema.**
El prototipo se **centra en la experimentación de los requerimientos del cliente que no se comprenden del todo.**

Modelo en espiral

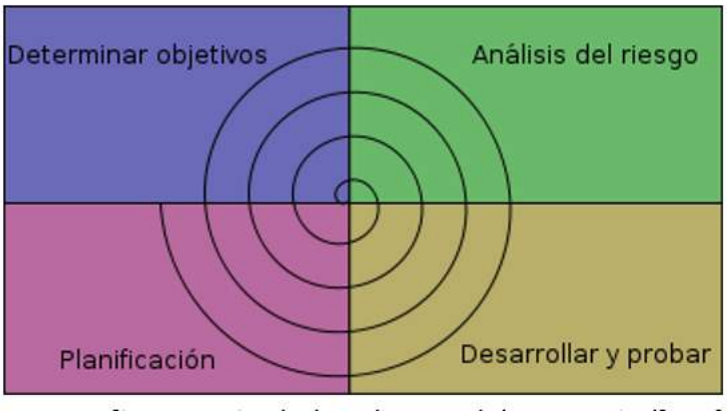
Este modelo, propuesto originalmente por BOEHM en 1976, es un modelo de proceso de ***software* evolutivo** donde se conjuga la naturaleza de construcción de prototipos con los aspectos controlados y sistemáticos del **modelo lineal y secuencial.**

Proporciona el potencial para el desarrollo rápido de versiones incrementales del software que no se basa en fases claramente definidas y separadas para crear un sistema.

En el modelo espiral, el *software* se desarrolla en una serie de versiones incrementales. Durante las primeras iteraciones, la versión incremental podría ser un modelo en papel o un prototipo; durante las últimas iteraciones, se producen versiones cada vez más completas del sistema diseñado.

El modelo en espiral se divide en un número de actividades de marco de trabajo, también llamadas “regiones de tareas”. Cada una de estas regiones está compuesta por un conjunto de tareas del trabajo llamado “conjunto de tareas”, que se adaptan a las características del proyecto que va a emprenderse. En todos los casos se aplican actividades de protección.

Figura 5: Modelo en espiral



Fuente: [imagen sin título sobre modelo en espiral], s. f., https://bit.ly/3oJcOvI

Se puede observar que son cuatro áreas bien definidas:

Definición de objetivos: en esta área, se establece la especificación del alcance del producto que se obtendrá al final de la iteración (catálogo de objetivos y requisitos). Se identificarán riesgos y se determinarán posibles alternativas de solución.

Análisis del riesgo: aquí se analizan los riesgos involucrados en el desarrollo del proyecto. Para identificar los riesgos y asumir acciones por tomar con los riesgos, se utiliza el ciclo de *deming*.

Desarrollo y prueba: esta área comprende las actividades de análisis, diseño, construcción e implantación de la versión del producto.

Planificación: aquí se realiza un estudio de la situación actual del sistema y del proyecto, a partir del cual se determina la necesidad o no de una nueva evolución; en el caso de que sea necesaria, se realiza su planificación.

El movimiento de la espiral, que amplía su amplitud radial con cada iteración, indica que se van construyendo versiones sucesivas del *software* cada vez más completas.

Uno de los puntos más interesantes del modelo es la introducción al proceso de desarrollo de las actividades de análisis de los riesgos asociados al desarrollo y a la evaluación por parte del cliente de los resultados del *software*.

A diferencia del modelo de proceso clásico, que termina cuando se entrega el software, el modelo en espiral puede adaptarse y aplicarse a lo largo de la vida del software de computadora. Una visión alternativa del modelo en espiral puede ser considerada examinando el eje de punto de entrada en el proyecto.

Las regiones de tareas que componen este modelo son seis.

Figura 6: Modelo en espiral típico (6 regiones)



Fuente: [imagen sin título sobre modelo en espiral típico (6 regiones)], s. f., https://bit.ly/2QIKFs2

A continuación, proveemos más información de cada región:

- **Comunicación con el cliente:** las tareas requeridas para establecer comunicación entre el desarrollador y el cliente.
- **Planificación**: las tareas requeridas para definir recursos, el tiempo y otras cuestiones relacionadas con el proyecto. Son todos los requerimientos.
- **Análisis de riesgos**: las tareas requeridas para evaluar riesgos técnicos y otras informaciones relacionadas con el proyecto.
- **Ingeniería:** las tareas requeridas para construir una o más representaciones de la aplicación.
- **Construcción y adaptación:** las tareas requeridas para construir, probar, instalar y proporcionar soporte al usuario.
- **Evaluación del cliente:** las tareas requeridas para obtener la reacción del cliente según la evaluación de las representaciones del *software* creadas durante la etapa de ingeniería e implementación en la etapa de instalación.

Ventajas del modelo en espiral

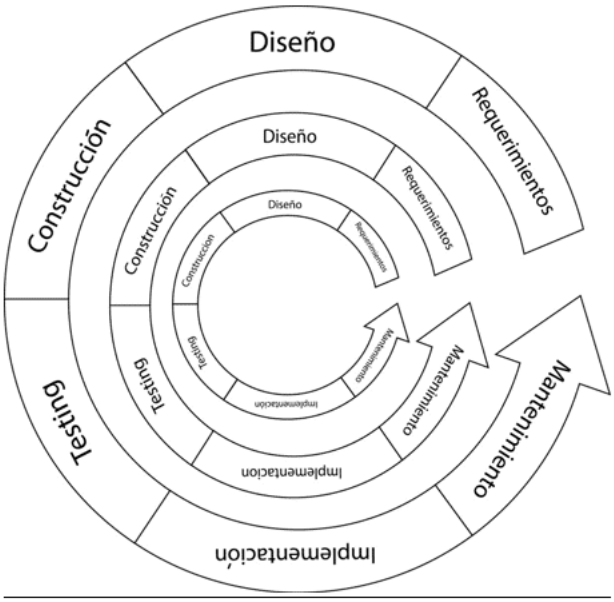
- Es el mejor modelo para el desarrollo de grandes proyectos.
- Se puede combinar con otros modelos de desarrollo.
- Puede comenzar con un proyecto con alto grado de incertidumbre.
- Tiene un bajo riesgo de retraso en caso de que se detecten errores.
- El error se soluciona en la rama de la espiral correspondiente.

Desventajas del modelo en espiral

- Se debe enfrentar el costo temporal que se suma en cada vuelta.
- Requiere comunicación con el usuario.
- En muchos casos, se considera complicada la realización del **análisis de riesgos**, ya que, en función del sistema de información, resulta muy complejo definir alternativas de solución y determinar con claridad cuál de ellas es la más apropiada.
- Es complicado realizar una planificación global del proyecto, puesto que eso dependerá de cómo evolucionen el proyecto, el producto y los usuarios en las distintas iteraciones. Se puede establecer una planificación inicial, con una serie de evoluciones, pero existe cierta incertidumbre respecto de hasta dónde se podrá llegar con el presupuesto inicial.

Ciclo de vida SDLC

Figura 7: Ciclo de vida SDLC



Fuente: elaboración propia.

SDLC es una metodología que basa el desarrollo de un software en varias etapas. Estas etapas varían según el autor o el modelo que se estén usando, aunque la mayoría de los autores concuerdan que el modelo cascada o

waterfall (en inglés) tiene las etapas básicas del SDLC y consideran así el modelo en cascada como el padre de todos los modelos de SDLC. Estas etapas son las siguientes:

- Análisis de requerimientos
- Diseño
- Codificación o programación
    - *Testing*
    - Implementación
    - Mantenimiento

Etapas

1

Primera etapa: análisis de requerimientos

Esta etapa se enfoca completamente en la recopilación de información del sistema que se va a desarrollar y está conformada por cuatro subetapas:

1) Obtención de requerimientos

En esta fase se obtiene la información sobre el proyecto: el cliente y el desarrollador se reúnen, y se tocan temas con respecto al proyecto para conseguir toda la información necesaria para su desarrollo.

2) Análisis de la información

Una vez reunida toda la información referente al proyecto, esa información se clasifica la que es realmente útil y la que no impacta en el proyecto en absoluto.

3) Especificación de la información

La información ya analizada es plasmada en un documento llamado SRS (*Software Requirement Specification* o especificación de los requerimientos de software). En este documento se plasma todo lo que se va a desarrollar, se mapean los requerimientos del cliente y se crea otro documento llamado RTM (*Requirements Traceability Matrix* o *matrix* de trazabilidad de requerimientos).

4) Validación de la información

En esta etapa, los documentos creados son mostrados al cliente, y el cliente los aprueba o desaprueba. Si es necesario, los documentos se modifican o se crean de nuevo; para su modificación o recreación, es necesario pasar nuevamente por las cuatro fases.

2 **Segunda etapa: diseño**

En esta fase, los documentos creados en la primera etapa, SRS y RTM, son traducidos a documentos de diseño. Dentro de todos los documentos de diseño que se pueden realizar, es necesario realizar documentos de alto y bajo nivel.

1. Diseño de bajo nivel

En estos documentos se plasma el algoritmo o la lógica que se va a usar en el proyecto. Es decir, estos documentos describen a fondo la funcionalidad del sistema. Los documentos más utilizados son los siguientes:

- Pseudocódigo
- Diagrama de clases

2. Diseño de alto nivel

En estos documentos se plasma, mediante diagramas, el flujo de la información y el modo en que la información va cambiando de proceso a proceso. Los diagramas utilizados varían de empresa a empresa, pero los más usados son los siguientes:

- Diagramas de contexto
- Diagramas entidad-relación
- Diagramas de flujo de datos
- Diagrama de transición de estados

3 **Tercera etapa: construcción. Codificación o programación**

La codificación o programación es la etapa en la que los documentos de diseño son traducidos al lenguaje de programación.

Esta fase es quizás la más simple, debido a que solo hay que crear o construir lo que en el diseño se pensó. Aunque no solo se codifica, se tiene que validar y verificar el código.

La validación es también llamada *unit testing o testing* de unidad. El programador verifica que el programa realice exactamente lo que se pide en el SRS, puesto que este documento constituye las reglas del negocio; en pocas palabras, el programador valida que el programa realice exactamente lo

que tiene que hacer. Esta validación puede hacerla el mismo programador (*desk check* o prueba de escritorio) o un colega (*peer review* o revisión de compañero).

En esta etapa se verifica que el código cumpla con las especificaciones o estándares, ya sean estos propios de la empresa que lo desarrolla, los estándares del cliente o estándares de ambos.

Se valida primero el programa, para certificar su funcionamiento, y después se da paso a la aplicación de los estándares de código, de interfaces o de cualquier estándar que aplique al proyecto en realización.

4

Cuarta etapa: *testing*

La etapa de pruebas es crucial en cualquier proyecto de desarrollo debido a que certifica la calidad del producto; y cuanto más calidad el producto tenga, más costoso será el producto. El QA team o equipo de calidad es el encargado de llevar a cabo esta fase, ya que un programador no lo puede realizar, no por su falta de habilidad o conocimiento, sino debido a una cuestión de objetividad. Para ejemplificar esto, voy a repetir lo que dijo el primer entrenador que tuve:

Para poder llevar a cabo el *testing*, se necesitan dos cosas:

- La versión beta del programa

Un documento donde se certifique que se alcanzó un 100 % de cobertura del código

Una vez que los *testers* tengan estos documentos en sus manos, el líder de proyecto realiza los siguientes documentos:

- *Strategy plan*
- *Test plan*
- *Traceability matrix*

Y en base a estos dos documentos, los testers crean los siguientes documentos:

- *Test data*
- *Test suites*
- *Test cases*
- *Test scripts*

Concluida la fase de *testing*, el *leader*, sobre la base de la información generada, genera un *test execution repor*t o reporte de ejecución de pruebas, el cual será utilizado por el líder de proyecto para decidir si el programa está listo para su implementación o si aún necesita algunas mejoras o correcciones.

Strategy plan

Este documento es usado para determinar lo que se va a medir, el ambiente de pruebas necesario y los criterios para detener y reanudar la etapa del *testing*. El documento *Strategy plan* requiere la firma tanto del líder como del cliente para que ambos estén al tanto de la estrategia que se va a seguir durante esta fase.

Test plan

En este documento se determina el alcance de lo que se va a probar. Qué entra y qué no en este apartado es importante para dedicar el tiempo que se necesita a las partes que realmente lo necesitan. Asimismo, se definen los tipos de pruebas que se realizarán y que se necesitan para realizar cada prueba. Un cronograma es definido para determinar los tiempos que se dedicarán a cada prueba. Al igual que el documento anterior, este también necesita de la firma tanto del líder del proyecto como del cliente para que ambos acuerden los alcances de esta etapa.

Traceability matrix

Este documento se crea para tener un *mapping* o mapeo de los test con los requerimientos, para determinar su prioridad, severidad y los tiempos necesarios para cada test, y, en caso de que surjan problemas o dudas, verificar más rápidamente el requerimiento que es cubierto por el *tester* con dudas.

Test data

En este documento se define la información que será usada para esta etapa, es decir, la información necesaria para ejecutar un *testing*.

Test suite

Este documento es opcional, ya que solo contiene un recopilado de ciertos *testings* que cumplen con el escenario de pruebas, es decir, un escenario específico, una acción que se debe realizar en el sistema. Por ejemplo, podemos crear un escenario de prueba para la acción de agregar otro más, para buscar, y así sucesivamente.

Aunque, en teoría, los *Test suites* son creados para específicas acciones, a veces, se crean para ciertas regiones; es decir, si el sistema va a ser usado tanto en Jalisco como en Guanajuato, se puede crear un Test suite para Jalisco y otro más para Guanajuato.

Test case

Este es el documento en el cual se especifica qué acción o movimiento se va a probar. En esencia, se puede decir que es el documento más pequeño que se crea en esta fase, ya que se describe una situación específica para probar.

Test script

Este documento se puede crear por separado o dentro del Test case, puesto que define la serie de pasos por seguir para poder ejecutar un determinado *testing*.

El Test script siempre va ligado a uno o varios *testings*

5

Quinta etapa: implementación

En esta etapa, el programa ya se encuentra listo. Se ha encontrado y se ha corregido el 95 % o más de los defectos, y el programa se encuentra preparado para su implementación en el ambiente de producción.

Durante esta etapa, se generan varios documentos, entre los que se destacan los siguientes:

· *Roll-Back plan* o Plan de revertir

· *Training* o Capacitación

· *Ralease Notes* o Notas de liberación

Roll-back plan

Se trata de un documento en el cual se indica qué pasos se deberán seguir en caso de que la instalación o la implementación hayan fracasado para "desinstalar" y regresar el ambiente de producción al momento anterior a su instalación

Training

El training no es un documento en sí, pero se genera documentación necesaria para que el usuario final entienda el funcionamiento del programa.

En algunos casos, la capacitación es dada por un *tester* durante cierto tiempo, donde este demuestra cómo se maneja la aplicación y les enseña a los usuarios cómo se usa correctamente.

Release notes

Es un documento en el cual se indican todas las notas referentes al *release* actual, pueden ser notas, comentarios o incluso un manual en el que se indique cómo resolver dudas o problemas.

6

Sexta etapa: mantenimiento

En esta etapa, el programa es tomado y, según el caso, se actualiza, adapta, corrige o mantiene. Es decir, se toma el programa, se vuelve a codificar, se vuelve a testear y se vuelve a dar la implementación según los requerimientos o necesidades actuales del sistema.

Reutilización del *software*

Las etapas de requerimientos y la de validación son comparables a las de otros procesos. Las etapas del proceso son las siguientes:

- Análisis de componentes
- Modificación de requerimientos
- Diseño de sistemas con reutilización
- Desarrollo e integración

Análisis de componentes

Teniendo una especificación de requerimientos, se buscan los componentes para esta especificación. En la mayoría de los casos, no existe la concordancia exacta, y los componentes que se utilizan solo proporcionan parte de la funcionalidad requerida.

Modificación de requerimientos

En esta etapa, los requerimientos se analizan utilizando información acerca de los componentes que han descubierto.

Si las modificaciones no son posibles, la actividad de análisis de componente se puede llevar a cabo nuevamente para buscar soluciones alternativas.

Diseño de sistemas con reutilización

En esta fase, se diseña o se reutiliza un marco de trabajo para el sistema.

Los diseñadores tienen en cuenta los componentes que se reutilizan, y se organiza el marco de trabajo para que los satisfaga. Si los componentes reutilizables no están disponibles, es posible que se tenga que diseñar un nuevo *software*.

Desarrollo e integración

Para crear el sistema, se desarrolla el software que no se puede adquirir y se integran los componentes y los sistemas.

En este modelo, la integración de sistemas, más que una actividad separada, es parte del proceso de desarrollo.

CONTINUAR

Metodología de elicitación

El objetivo de esta metodología es la definición de las tareas por realizar, los productos por obtener y las técnicas por emplear durante la actividad de elicitación de requisitos de la fase de ingeniería de requisitos del desarrollo de *software*.

En esta metodología, se distinguen dos tipos de productos: **los productos entregables** y **los productos no entregables o internos**. Los productos entregables son aquellos que se entregan oficialmente al cliente como parte del desarrollo en fechas previamente acordadas, mientras que los no entregables son productos internos al desarrollo que no se entregan al cliente.

Los elementos que componen la metodología son los siguientes:

Para obtener la Información, tenemos diferentes técnicas:

Técnicas de elicitación

Introducción

Para recopilar información sobre el negocio del cliente o sobre los requisitos del sistema que se va a desarrollar, tenemos diferentes técnicas:

1 **Entrevista**

- Es una técnica para la recolección de datos.
- Es una conversación dirigida con un propósito específico y consiste en preguntas y respuestas.

En una entrevista, se pretenden conocer tanto las opiniones como los conocimientos del entrevistado acerca del estado actual.

Planificación de una entrevista

1. Lectura de antecedentes

2. Establecimiento de los objetivos de la entrevista
3. Selección de los entrevistados
4. Preparación del entrevistado
5. Selección del tipo y estructura de las preguntas

Lectura de antecedentes

Consultamos y comprendemos el mayor número posible de antecedentes de los entrevistados y de su organización. Es importante conocer de antemano las necesidades de la empresa para aprovechar al máximo el tiempo de la entrevista.

Establecimiento de los objetivos de la entrevista

Establecemos los objetivos de la entrevista sobre la base de los antecedentes que consultamos y nuestra experiencia en particular.

Es importante tener un objetivo bien preciso cuando salimos a r**ealizar una entrevista y respetar el tiempo y necesidad del entrevistado.**

No debemos imponer nuestros objetivos. Si lo hacemos, la entrevista no tiene sentido.

Más adelante conversaremos sobre la calidad de la información.

Selección de los entrevistados

Es normal que se entrevisten gerentes, usuarios, administrativos, auditores, programadores, etcétera.

Es indispensable comenzar a relevar a las autoridades de la empresa (gerentes, ejecutivos, jefes), respetando las jerarquías.

La opinión de estas autoridades es fundamental, debido a que son quienes tienen poder de decisión y definen los objetivos de una organización.

Preparación del entrevistado

Es necesario preparar a la persona que entrevistará, avisarle con tiempo para que pueda pensar acerca de la entrevista.

Es conveniente recordarle al entrevistado unas horas antes el compromiso asumido.

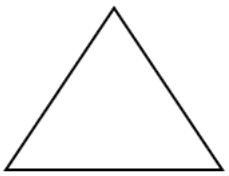
El tiempo asignado a las entrevistas debe ser de 45 minutos a una hora. No es aconsejable extenderse más del plazo previsto. Muchas veces, con niveles gerenciales, la entrevista dura un tiempo muy corto.

Estructura de preguntas

- **Estructura pirámide**
- **Estructura embudo**
- **Estructura diamante**

Estructura pirámide

Figura 8: Pirámide

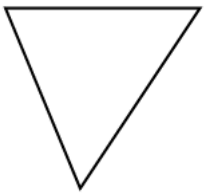


Fuente: elaboración propia.

Se comienza con preguntas cerradas y se termina con preguntas abiertas

Estructura embudo

Figura 9: Embudo

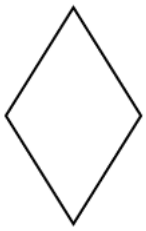


Fuente: elaboración propia.

Se comienza con preguntas abiertas y se termina con preguntas cerradas.

Estructura de diamante

Figura 10: Diamante



Fuente: elaboración propia.

Se comienza con preguntas cerradas, luego se sigue con preguntas abiertas y se termina con preguntas cerradas.

Tipos de preguntas

- Abiertas
- Cerradas

Preguntas abiertas

- ¿Qué opina acerca de las computadoras para gerentes?
- ¿Cómo toma sus decisiones sobre la programación?
- ¿Cuál es su opinión del sistema actual?
- ¿Cómo ve los objetivos del departamento?

Ventajas de las preguntas abiertas

- Simplifican las cosas al usuario.
- Permiten al entrevistador seleccionar un vocabulario del entrevistado, lo que pone de manifiesto su educación, valores y creencias.
- Revelan nuevas alternativas para preguntas no consideradas.
- Proporcionan una gran fuente de detalle.
- Hacen más interesante la entrevista.

- Permiten mayor espontaneidad.
- Constituyen una alternativa cuando el entrevistado no está preparado.

Desventajas de las preguntas abiertas

- Pueden generar demasiada información irrelevante.
- Es factible la pérdida de control de la entrevista.
- Se pueden perder los objetivos reales de la entrevista.

Preguntas cerradas

- Son la alternativa a las preguntas abiertas.
- Limitan las posibles respuestas del entrevistado.

El entrevistado puede responder lo siguiente:

- Uno, diez.
- Ninguno
- Sí/No
- Verdadero/Falso

O puede elegir opciones.

Ejemplos con números:

> Pregunta: ¿Cuántas personas subordinadas tiene a su cargo?
>
> Respuesta: Uno, veinte, ninguno.
>
> Pregunta: ¿Cuántos informes genera en un mes?
>
> Respuesta: Ninguno, uno, diez.

Ejemplos para elegir opciones

De las siguientes opciones, ¿cuáles son las más valiosas?:

1. Formas de reclamo llenadas por clientes
2. Relación cara a cara con clientes

3. Devolución de mercaderías

De las siguientes opciones, ¿cuál es la más importante?:

1. Reclamo por teléfono
2. Reclamo por cartas
3. Reclamos judiciales

Ejemplos con Sí/No

Existen técnicas nuevas en donde se reemplazan las preguntas con signo de interrogación por otras concretas.

Saludo al cliente: Sí/No

- Hubo una sonrisa.
- El saludo fue sincero
- Hubo contacto visual.
- **Atención del pedido:**
  - Está familiarizado con el menú.
  - El cliente hizo el pedido una sola vez.
  - Los pequeños pedidos fueron memorizados.
  - Fue sugerida la venta.
- **Preparación del pedido**
  - El orden de preparación del pedido fue el apropiado.
  - La bolsa de asado se utilizó en primer lugar.
  - La cantidad de hielo fue adecuada.
  - Las bebidas se prepararon en el orden apropiado.
  - Las bebidas se llenaron hasta el nivel apropiado.
  - Se les puso la tapa a los vasos.
  - Se limpiaron los envases.
  - Observó el orden en el café.
  - Se llenaron las tazas en el nivel justo.
- **Presentación del pedido**
  - Está debidamente empaquetado.
  - El extremo de la bolsa se ha doblado dos veces.

- Se utilizaron fuentes de plástico internas.
- Los alimentos se prepararon en forma adecuada.

- **Petición y recepción de pagos**
  - La cantidad se dice en voz clara y audible.
  - La cantidad recibida fue claramente escrita.
  - El cambio se contó con voz clara y alta.

- **Agradecimiento al cliente y sugerencia de repetir la visita**
  - Hay siempre un agradecimiento.
  - Las gracias son sinceras.
  - Hay un contacto visual.
  - Se sugiere al cliente que vuelva.

Ventajas de las preguntas cerradas

- Ahorran tiempo.
- Facilitan la comparación entre entrevistas.
- Llegan al punto de interés.
- Mantienen el control de la entrevista.
- Cubren rápidamente diferentes aspectos.
- Obtienen datos significativos.

Consejos para tener en cuenta en una entrevista

- **Evite las preguntas tendenciosas**

  "Usted está de acuerdo, como los otros gerentes, con que el control de inventarios debe estar computarizado" se podría reemplazar por "¿Qué piensa del control computarizado de inventarios?".

- **Evite las preguntas dobles**

  ¿Qué decisiones se pueden tomar en un día normal y cómo las toma usted?

2 **Observación directa**

Se observan directamente las prácticas profesionales que se realizan habitualmente en la organización para la que se va a desarrollar el *software*. Antes de celebrar una sesión de observación in situ, se deben escoger un conjunto de prácticas representativas del resto que se lleven a cabo con una frecuencia relativamente alta o que presenten cierta complejidad de comprensión. Además, se debe intentar que los resultados de la práctica profesional sean observables en el entorno real de trabajo.

Existen algunas variantes de la observación in situ, como la inmersión dentro de la organización para la que se va a desarrollar el software o la realización de periodos de aprendizaje por parte de los ingenieros de requisitos, en las que la observación pasiva se cambia por una participación en los procesos por observar como si se fuera un miembro más del personal de la organización bajo estudio.

Es una de las técnicas de las que mejores resultados se obtienen porque se observa el funcionamiento de cada proceso.

3 ***Joint Application Development* (reuniones integradas)**

El JAD (*Joint Application Development*) consiste en organizar reuniones integradas por miembros del equipo de desarrollo y miembros de la organización para la que se va a desarrollar el sistema *software*.

Durante una sesión de JAD, el coordinador plantea las cuestiones que se van a discutir para determinar los **objetivos y los requisitos generales del sistema** que se va a desarrollar, y los participantes contestan a dichas cuestiones.

Si quedan temas abiertos, el coordinador de la sesión debe documentar esto para programar otra reunión en la que se aborden dichos temas.

Si ya se dispone de los modelos de negocio del sistema que se va a desarrollar, este material puede resultar útil para consultar y aclarar algunos aspectos durante la sesión.

4 **Tormenta o lluvia de ideas**

La tormenta o lluvia de ideas es una técnica de reuniones en grupo cuyo objetivo es la generación de ideas en un ambiente libre de críticas o juicios.

Como técnica de recopilación de información de requisitos, la tormenta de ideas puede ayudar a generar una gran variedad de vistas del problema y a formularlo de diferentes formas, sobre todo, al comienzo del proceso de ingeniería de requisitos, cuando estos últimos están todavía muy difusos y poco claros

5 **Talleres de trabajo**

Los talleres de trabajo son reuniones integradas por clientes, usuarios y parte del equipo de desarrollo. La principal particularidad es que el coordinador debe dirigir la sesión intentado limitar la profundidad con que se abordan los requisitos según el nivel establecido a priori para la sesión.

Por ejemplo, durante la exploración inicial de casos de uso, el tiempo límite por caso de uso puede ser de 30 minutos y evitar así entrar en un nivel excesivo de detalle antes de tiempo.

Las principales reglas durante el desarrollo de los talleres son las siguientes: comenzar y terminar a la hora prevista; mantener una sola conversación en cada momento; no interrumpir al que esté hablando; y centrar los comentarios y críticas en el asunto que sea, sin incidir en alguien en concreto.

6

Cuestionarios:

El cuestionario es dirigido a un número menor de personas de las que necesitamos obtener información. Por ejemplo, en una empresa en la que necesito saber la opinión de 20 personas sobre el funcionamiento de un área determinada, puedo recurrir a un cuestionario compuesto por preguntas abiertas y tener la opción de alguna pregunta cerrada

El embudo es una técnica que se utiliza en el cuestionario.

- ◯ Falso.
- ◯ Verdadero.

SUBMIT

Un cuestionario solamente tiene preguntas abiertas.

- ◯ Falso.
- ◯ Verdadero.

SUBMIT

Las preguntas cerradas tienen algunas ventajas. Puedes identificar las que son correctas de la siguiente lista:

- [ ] Ahorran tiempo.
- [ ] Facilitan la comparación entre entrevistas.
- [ ] Llegan al punto de interés.
- [ ] Son simples.

SUBMIT

En un ciclo de vida puede intervenir el cliente a medida que se desarrolla. Puedes identificar el ciclo de vida en la siguiente lista:

- ( ) Ciclo de vida evolutivo.
- ( ) Ciclo de vida en cascada.
- ( ) Ciclo de vida en espiral.
- ( ) Ciclo de vida SDLC.
- ( ) Ciclo de vida secuencial.

SUBMIT

CONTINUAR

Referencias

[Imagen sin título sobre análisis y definición de requerimientos], (s. f.). Recuperado de http://www.gml-2012.de/ausstellung/pieper.pdf

[Imagen sin título sobre modelo en espiral típico (6 regiones)], (s. f.). Recuperado de http://modeloespiral.blogspot.com/2009/08/modelo-tipico-de-seis-regiones.html

[Imagen sin título sobre modelo en espiral], (s. f.). Recuperado de https://upload.wikimedia.org/wikipedia/commons/3/39/ModeloEspiral.svg

[Imagen sin título sobre requerimientos del usuario], (s. f.). Recuperado de https://www.pinterest.es/pin/392024342555284714/

CONTINUAR

Revisión de Módulo

Hasta acá aprendimos

Ingeniería de requerimientos

En el desarrollo de la lectura identificamos los diferentes tipos de requerimientos existentes, las diferentes formas de documentar los requerimientos, la estructura de un SRS-830 y los conceptos detallados de un caso de uso.

Administración de requerimientos

En el desarrollo de la lectura estudiamos los conceptos de la administración de requerimientos, la planeación de la administración de requerimientos y los conceptos de administración del cambio en los requerimientos.

Validación de requerimientos

En el desarrollo de la lectura identificamos los diferentes elementos que componen la validación de requerimientos, teniendo en cuenta el desarrollo de diferentes tipos de verificación y pruebas disponibles.

Ciclos de vida. Técnicas de elicitación

A lo largo de la última lectura identificamos cada uno de los ciclos de vida que se utilizan en el desarrollo de un sistema, partiendo del ciclo de vida en cascada, siguiendo con el ciclo de vida de desarrollo incremental o evolutivo, el ciclo de vida en espiral.

CONTINUAR